

松灵机器人产品Cobot Kit 复合移动机器人开发套件用户指导手册



松灵机器人产品Cobot Kit 复合移动机器人开发套件用户指导手册

- 1 产品简介
- 2 主要配置清单以及参数说明
 - 2.1 主要配置
 - 2.2 主要配件介绍
- 3 开发前准备
 - 3.1 下载远程桌面工具
 - 3.2 连接控制机械臂运动
 - 3.3 使用ar_track_alvar实现视觉定位追踪
 - 3.4 使用Gmapping构建地图
 - 3.5 使用rtabmap算法构建地图
 - 3.6 使用move_base导航
 - 3.7 使用smach库实现任务状态切换
 - 3.8 使用MoveIt! 控制移动+臂仿真模型
- 4 功能使用
 - 4.1 启动机械臂
 - 4.2 启动摄像头
 - 4.3 启动机械爪
 - 4.4 启动agx_cr5_pick节点
 - 4.5 启动smach状态机节点
 - 4.6 启动雷达
 - 4.7 启动建图
 - 4.8 启动导航
 - 4.9 录制两导航点的坐标位置
 - 4.10 移动抓取演示
 - 4.11 原地抓取和放置演示
- 5 常见问题的处理与解答
- 6 其他说明

1 产品简介

Agile 复合移动机器人系列开发套件是松灵机器人专为行业科研教育应用客户研发开发的高级开发套件，面对科研教育对机器人产品越来越灵活的，该套装基于松灵机器人ROS生态体系统，集成高性能工控、高精度LiDAR、多传感器、多自由度机械臂、视觉感知搭配于一身。套件包含了多线激光雷达自主导航、机械臂moveIt运动控制规划、视觉识别、机械臂自主抓取等功能于一体，解决客户在面对移动机器人在复杂应用场景下的技术复杂度高、集成难度高的特征，给客户带来极致的用户体验。产品包含完整的技术手册与配套的技术文档、代码采用全开源的形式，降低客户的应用和学习难度。产品套件可以广泛用于农业、智能制造、教育实训、科研探索等方向。

2 主要配置清单以及参数说明

2.1 主要配置

套件版本	Cobot Kit
底盘移动平台	Ranger mini v3
机械臂	Dobot CR5
机械臂夹爪	大寰AG 95
工控机	x-7010(i7 16G 256SSD)
视觉传感器	RealSense D435
激光雷达传感器	RS-Helios-16P
屏幕	高清显示屏
路由器	B316 4G路由器

2.2 主要配件介绍

- SCOUT 产品介绍

SCOUT是一款全能型行业应用的轮式底盘车。它具有操作简单灵敏，开发空间大，适应多种领域开发应用，独立悬挂系统，重载避震，爬坡能力强，可爬楼梯等特点，可用于巡检勘探、救援排爆、特种拍摄、特种运输等特种机器人开发，解决机器人移动方案。



项目	指标
长×宽×高（mm）	720×500×345
轴距（mm）	494
前 / 后轮距（mm）	364
整备重量（Kg）	75
电池类型	磷酸铁锂
电池参数	48V24AH
电压	24V
动力驱动电机	350W×4
转向驱动电机	100W×4
驻车形式	电子刹车
转向形式	四轮四转
悬挂形式	独立悬挂加摆臂
遥控器	2.4G /极限距离1KM
通讯接口	CAN

• **Dobot 机械臂介绍**

Dobot系列机器人是松灵机器人根据科研教育行业严选出来的一款工业级轻量级协作机器人产品，产品采用关节模块化设计，使用面向开发者层面的机器人系统。用户可根据Dobot平台提供的应用程序接口，开发属于自己的机器人控制系统；此外，Dobot机器人配有专用的可编程操作界面，用户可通过此界面实时观察机器人的运行状态，对机器人进行诸多控制设置，也可脱机进行离线仿真，极大地提升了实际应用的工作效率。同时我们根据科研教育行业的特点，适配ROS，支持Moveit等开源方案支持。



机器人类型	Dobot CR5
重量	24 kg
有效负载	5 kg
延伸	924 mm
关节范围	-175° ~ +175°
关节速度	1-3关节: 150°/s 4-6关节: 180°/s
工具速度	≤2.8 m/s
重复定位精度	± 0.02 mm
占地面积	Ø172 mm
自由度	6个旋转关节
标准控制柜尺寸（长*高*宽）	727mm x 623mm x 235mm
I/O 电源	控制柜中为24V 3A，工具中为 0V/12V/24V 0.8A
通信协议	Ethernet、Modbus - RTU/TCP
接口与开放性	SDK（支持C\C++\Lua\Python开发）、支持ROS系统、API
编程	在 Dobot Windows 客户端 DobotSCStudio 上进行
噪声	噪声小
IP防护等级	IP 54
功耗	运行典型的程序时大约为200W

• X-7010工控机介绍

X-7010是一款模块化组合的高性能超小工控机，针对移动机器人行业算力要求高，环境要求高的特征而定制的一款工业工控控制电脑。它采用Intel 平台，支持8/9th 35W高速处理器。它采用高效热管的大面积铝鳍和PWM风扇的主/被动双重散热设计，通过模具打造的全铝合金强固机身，保证其长寿命稳定运行。适用于智能机器人、无人驾驶、机器视觉、智慧城市等高运算要求领域。预装 Ubuntu 18.04， ROS Melodic（Full Desktop）版本，以及一些机器人开发常见得开发环境，实现开机即用。

- 具有巴掌大小紧凑超小机身；
- 支持Intel 8th桌面级高性能CPU；
- 搭载热管与智能风扇的主被动高效散热；
- 支持miniPCIE、NVME等多种加速卡扩展方案；
- 多路超高速专用串口，适配多种雷达应用；
- 多通道USB3.1 Gen2与双千兆网络的高速通讯；
- 坚固的模具成型的铝合金机身，符合车载振动冲击；
- -20~60℃宽温工作；



- **视觉传感器RealSense D435**

双目视觉传感器，在机器人视觉测量、视觉导航等机器人行业方向中均有大范围的应用场景和需求，目前我们甄选了了在科研教育行业常见的视觉传感器。英特尔实感深度摄像头 D435 配备全局图像快门和宽视野，能够有效地捕获和串流移动物体的深度数据，从而为移动原型提供高度准确的深度感知。



	型号	Intel Realsense D435
基本特征	应用场景	户外/室内
测量距离	约10米	
深度快门类型	全局快门/3um X 3um	
是否支持IMU	支持	
深度相机	深度技术	有源红外
FOV	86° x 57° (±3°)	
最小深度距离	0.105m	
深度分辨率	1280 x 720	
最大测量距离	约10米	
深度帧率	90 fps	
RGB	分辨率	1280 x 800
FOV	69.4° x 42.5° (±3°)	
帧率	30fps	
其他信息	尺寸	90mm x 25mm x 25mm
接口类型	USB-C 3.1	

• 雷达传感器

RS-Helios-16P 是 激光雷达中小巧而先进的一款产品。与同等价位的传感器相比，RS16 性价比更高，并且保留了 RoboSense 在激光雷达方面比较有突破性的一些主要特点，如实时、360°、三维坐标和距离、附带校准的反射率测量。

RS-Helios-16P 测量范围可达 150 m，功耗低（约 11 W），重量轻（约 0.99kg），体积小（Ø97.5mm x 100mm），具备双重返回性能，这些特点使它成为背包式测量、无人机挂载和其它移动设备的理想选择。

项目	参数
最大测距	150m
测距精度	±3cm
扫描速率	单次回波30万点/秒 双回波60万点/秒
垂直视角	-15°~+15°
垂直角分辨率	2°
扫描频率	10Hz~20Hz
安全等级	Class 1
重量	0.99kg
功耗	11W
电压	9V~32V
工作温度	-10℃~+60℃

• 机械夹爪

大寰机器人AG 95 电动夹爪拥有两个自适应平行机械 关节手指（后指代 关节手指每个 关节手指由多个连杆机构和一个弹簧组成。 关节手指可以与一个物体进行多达 5 个接触点的接触。 关节 手指采用欠驱动控制方式驱动，使得电机比 关节的总数要少。这种设计简化了抓取的控 制方式，使 关节手指 可以自动适应它们所抓取的物体形状。



最大推荐负载	3-5kg*
手指开合行程（编程可调）	0-95mm
抓持力（编程可调）	45-160N
最快手指开合速度	190mm/s
自身重量	1kg
手指重复定位精度	0.03mm
通讯协议	TCP/IP, USB2.0, RS485, I/O, CAN2.0A, EtherCAT（选配）
工作电压	24V DC±10%
工作温度范围	0~50℃

3 开发前准备

3.1 下载远程桌面工具

在机器人中的工控机中安装了一个远程桌面的工具，用户需要在自己的笔记本或者电脑上安装相应的工具，通过机器人上面的路由器远程操控机器人上的工控机。

（1）下载安装包

<https://www.nomachine.com/download>

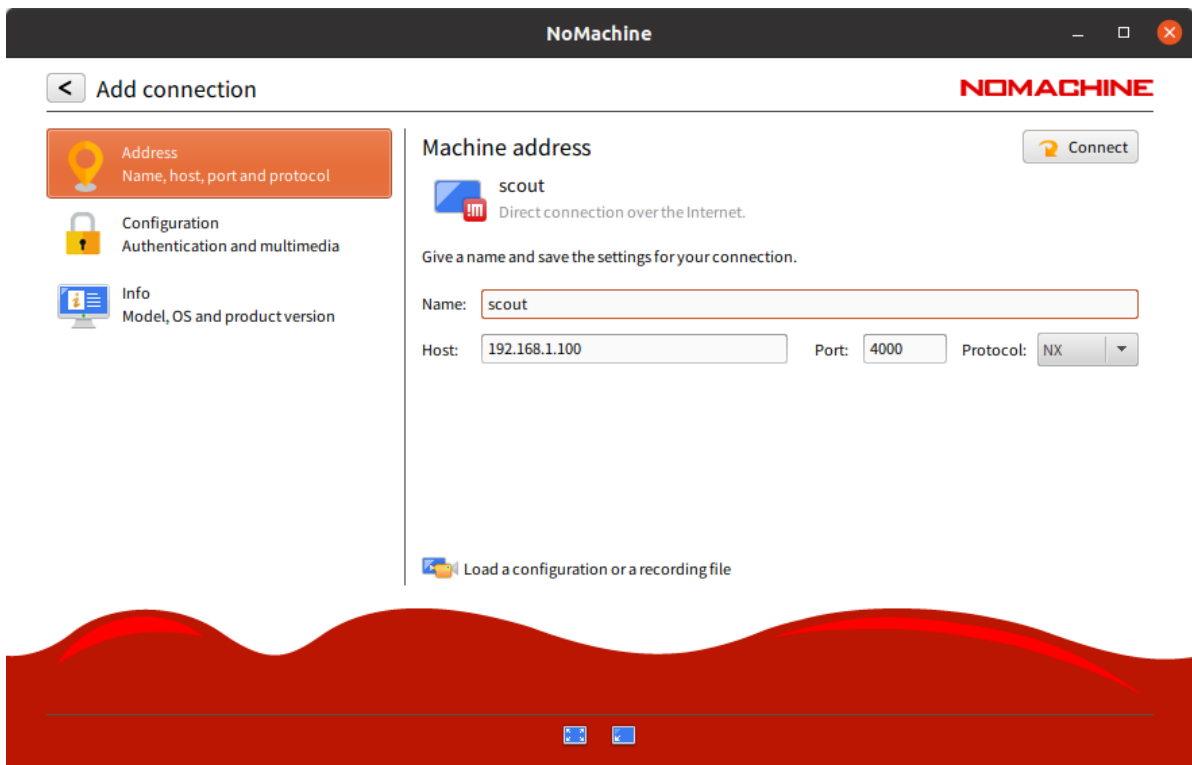
选择自己笔记本或者电脑对应的操作系统进行下载安装。

（2）使用方法

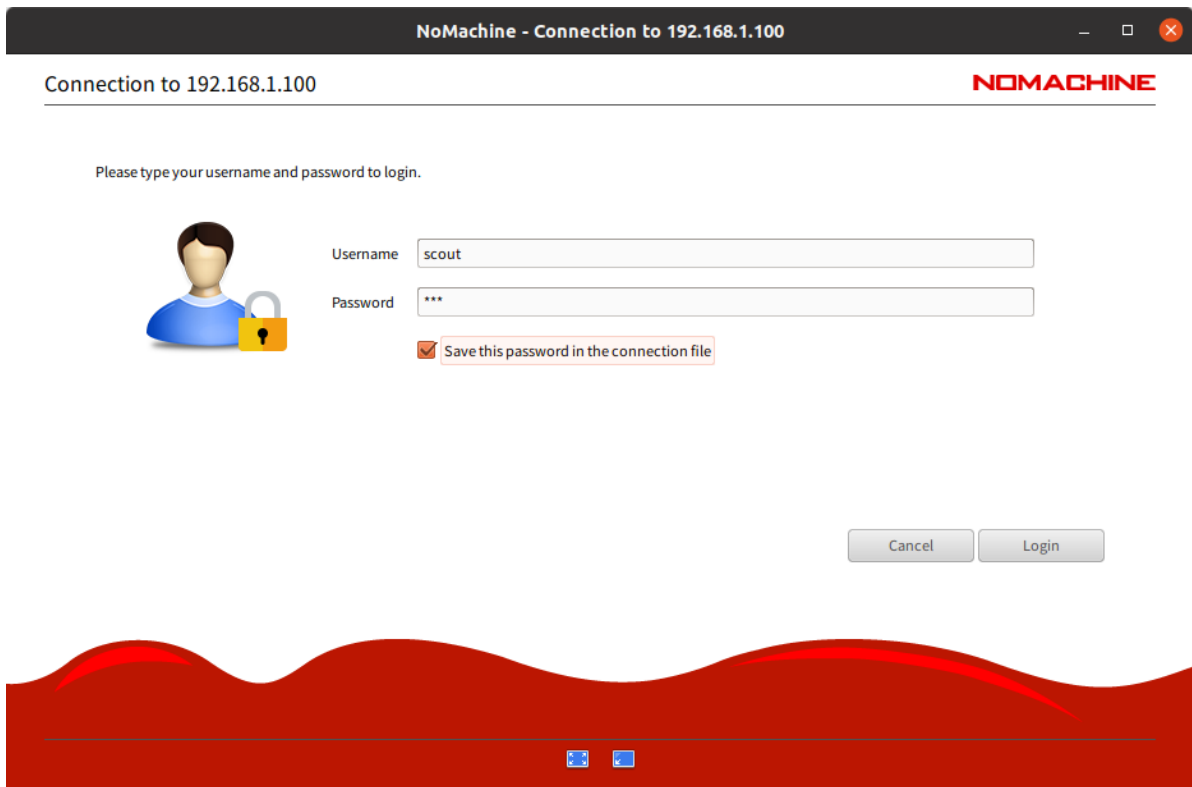
工控机上的用户：agilex，电脑的开机密码：agx。路由器名字：SY**，路由器的密码是12345678；路由器后台的登录用户和密码都是是admin。

首先用自己的电脑或者笔记本找到机器人上面的wifi，输入密码进行连接。

打开下载好的远程连接工具NoMachine，添加一个远程连接（图中Host改为工控机实际的IP）：



输入username: agilex和密码: agx, 勾选记住密码。



就可以连接到机器人上的工控机了。

3.2 连接控制机械臂运动

(1) 首先确保机械臂控制器已上电, 机械臂5轴部位面板蓝灯常亮, 此时打开终端, 运行命令:

```
$ roslaunch dobot_bringup bringup.launch robot_ip:=192.168.8.188
```

```
/home/scout/agilex_ws/src/moveit_config/scout_cr5/cr5_ros/dobot_bringup/launch/bringup...
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

scout@agilex:~$ roslaunch dobot_bringup bringup.launch robot_ip:=192.168.8.188
... logging to /home/scout/.ros/log/b59c5d14-f1de-11ec-b471-68eda453fdff/roslaun
ch-agilex-7842.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://agilex:40835/

SUMMARY
=====

PARAMETERS
* /cr5_robot/joint_publish_rate: 10.0
* /cr5_robot/robot_ip_address: 192.168.8.188
* /cr5_robot/trajectory_duration: 0.3
* /rostdistro: melodic
* /rosversion: 1.14.13

NODES
/
  cr5_robot (dobot_bringup/dobot_bringup)

auto-starting new master
process[master]: started with pid [7852]
ROS_MASTER_URI=http://localhost:11311

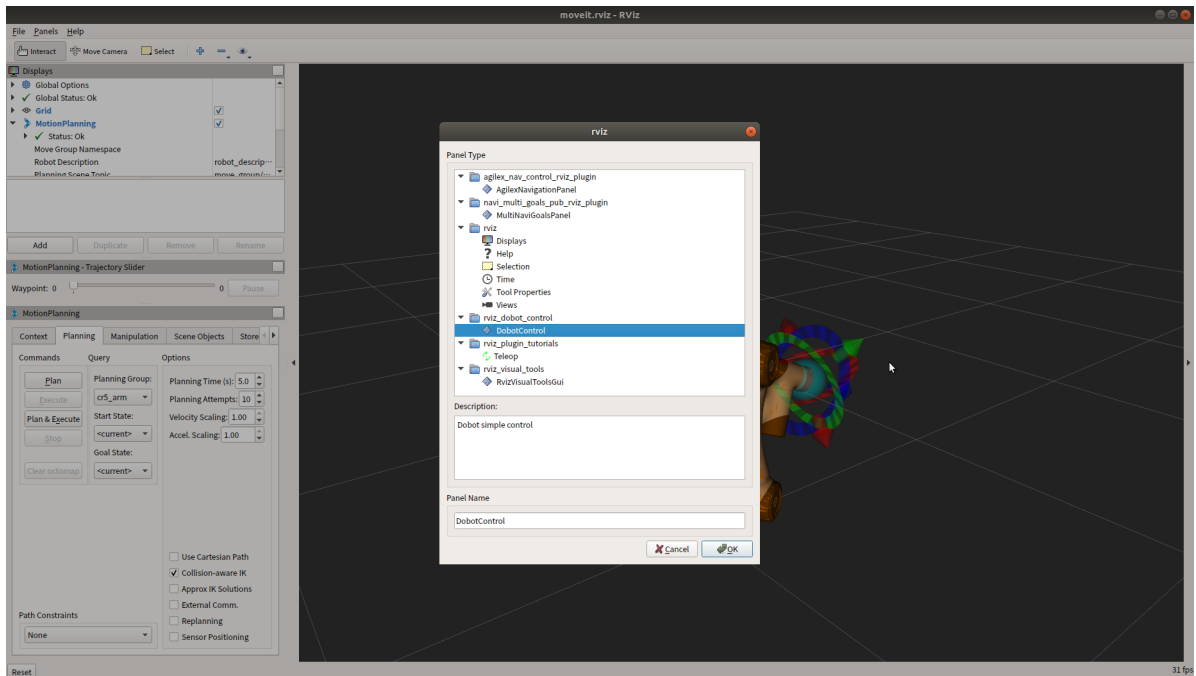
setting /run_id to b59c5d14-f1de-11ec-b471-68eda453fdff
process[rosout-1]: started with pid [7863]
started core service [/rosout]
process[cr5_robot-2]: started with pid [7869]
[ INFO] [1655869938.018800210]: trajectory_duration : 0.30
[ INFO] [1655869938.020108383]: 192.168.8.188:30004 : connect successfully
[ INFO] [1655869938.021027607]: 192.168.8.188:29999 : connect successfully
[ INFO] [1655869938.021876536]: 192.168.8.188:30003 : connect successfully
```

关于机械臂末端面板使用说明可参考《Dobot CR5硬件使用手册》第2.12节 机械臂末端按键说明。机械臂启动后，如果面板红灯常亮，可尝试运行命令 `rosservice call /dobot_bringup/srv/ClearError "{}"` 清除错误。如果依然红灯常亮，则需要使用Dobot Windows客户端软件进行操作，具体可参考其官方资料。

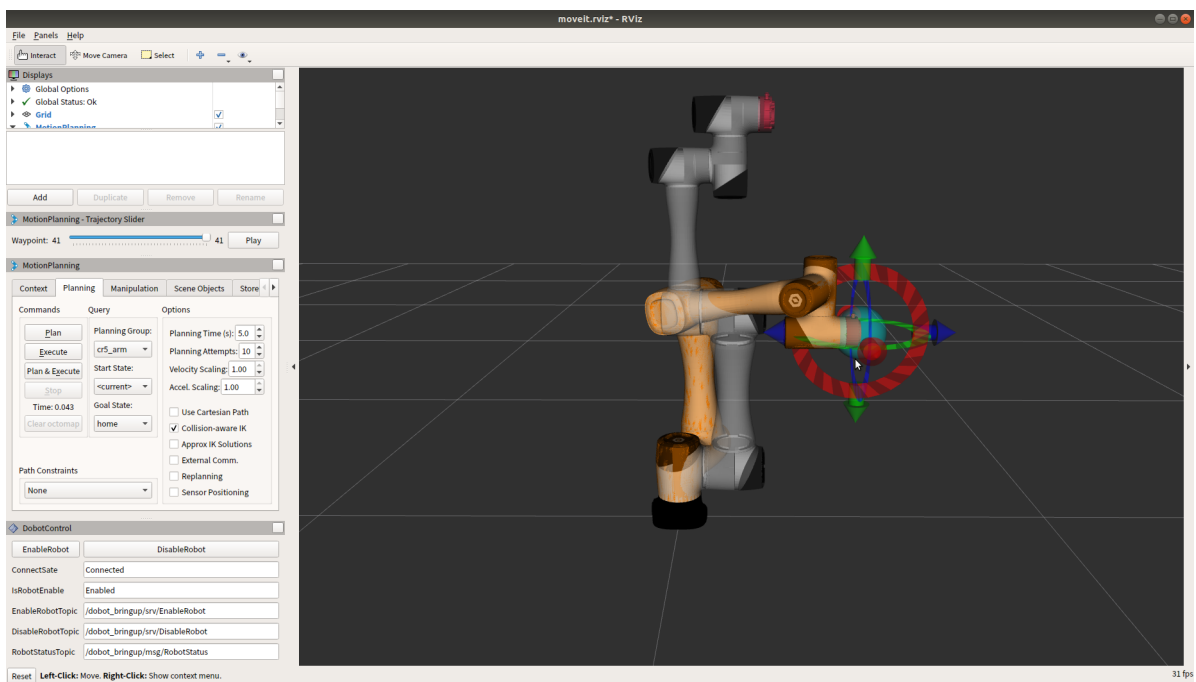
(2) 接下来启动MoveIt! 控制机械臂，运行命令：

```
$ roslaunch dobot_moveit moveit.launch
```

- 首先添加DobotControl操作面板，依次点击菜单栏Panels->Add New Panel，双击DobotControl完成添加。



- 点击DobotControl面板中的EnableRobot按钮，听见卡哒声，且机械臂末端模板由蓝灯常亮变为绿灯常亮，代表机械臂时能成功。
- 在RViz中拖拽机械臂末端到何时位置，然后点击Plan和Execute控制机械臂运动。



3.3 使用ar_track_alvar实现视觉定位追踪

3.3.1 功能简介

这个包是一个开源 AR 标签跟踪库 Alvar 的 ROS PACKAGE。

- (1) ar_track_alvar 有 4 个主要功能：生成不同大小、分辨率和数据/ID 编码的 AR 标签
- (2) 识别和跟踪单个 AR 标签的姿势，可选择集成深度相机的深度数据以获得更好的姿势估计。
- (3) 识别和跟踪由多个标签组成的组合姿势。这允许更稳定的姿态估计、对遮挡的鲁棒性以及多边形对象的跟踪。
- (4) 使用相机图像自动计算捆绑中标签之间的空间关系，这样用户就不必手动测量并在 XML 文件中输入标签位置来使用捆绑功能（目前不工作）。

Alvar 比 ARToolkit 更新、更先进, ARToolkit 一直是其他几个 ROS AR 标签包的基础。Alvar 具有自适应阈值处理以处理各种光照条件、基于光流的跟踪以实现更稳定的姿态估计, 以及改进的标签识别方法, 该方法不会随着标签数量的增加而显着减慢。

3.3.2 使用说明

(1) 生成标签

```
$ rosrn ar_track_alvar createMarker 0 -s 3.0
```

后面的数字表示生成标签的id号, 可以根据需要生成不同数字的标签

详细的参数设置如下图:

```
65535          marker with number 65535
-f 65535       force hamming(8,4) encoding
-1 "hello world" marker with string
-2 catalog.xml marker with file reference
-3 www.vtt.fi  marker with URL
-u 96          use units corresponding to 1.0 unit per 96 pixels
-uin           use inches as units (assuming 96 dpi)
-ucm           use cm's as units (assuming 96 dpi) <default>
-s 5.0         use marker size 5.0x5.0 units (default 9.0x9.0)
-r 5           marker content resolution -- 0 uses default
-m 2.0         marker margin resolution -- 0 uses default
-a            use ArToolkit style matrix markers
-p            prompt marker placements interactively from the user
```

(2) 打印标签

打印标签时注意尺寸, 默认使用3x3 cm的标签, 如果打印尺寸和默认尺寸不同, 请在
~/your_workspace/src/open_manipulator_perceptions/open_manipulator_ar_markers/launch/agx_ar_pose.launch文件中修改标签尺寸。

```
<arg name="user_marker_size"    default="3"/>
```

在使用过程中默认0号标签贴在物体上, 2号标签贴在物品摆放平台上。

(3) 运行识别功能

用usb3.0的线连接摄像头与电脑, 运行下面指令:

```
$ roslaunch agilex_cr5 agx_ar_pose.launch
```

其agx_ar_pose.launch文件的结构如下图所示, 详细参数参照下表:

```
<?xml version="1.0" ?>
<launch>
  <arg name="x"           default="0"/>
  <arg name="y"           default="0"/>
  <arg name="z"           default="0"/>
  <arg name="roll"        default="0"/>
  <arg name="pitch"       default="0"/>
  <arg name="yaw"         default="0"/>
  <arg name="r_x"         default="0"/>
  <arg name="r_y"         default="0"/>
  <arg name="r_z"         default="0"/>
  <arg name="r_w"         default="0"/>
  <arg name="parent_link" default="wrist3_Link"/>
  <arg name="serial_no"   default="" />
  <arg name="marker_frame_id" default="_color_frame"/>
  <arg name="user_marker_size" default="3"/>
  <arg name="use_quaternion" default="false"/>
  <arg name="camera_model" default="realsense_d435" doc="model type [astra_pro, realsense_d435, raspicam]"/>
  <arg name="camera_namespace" default="camera"/>

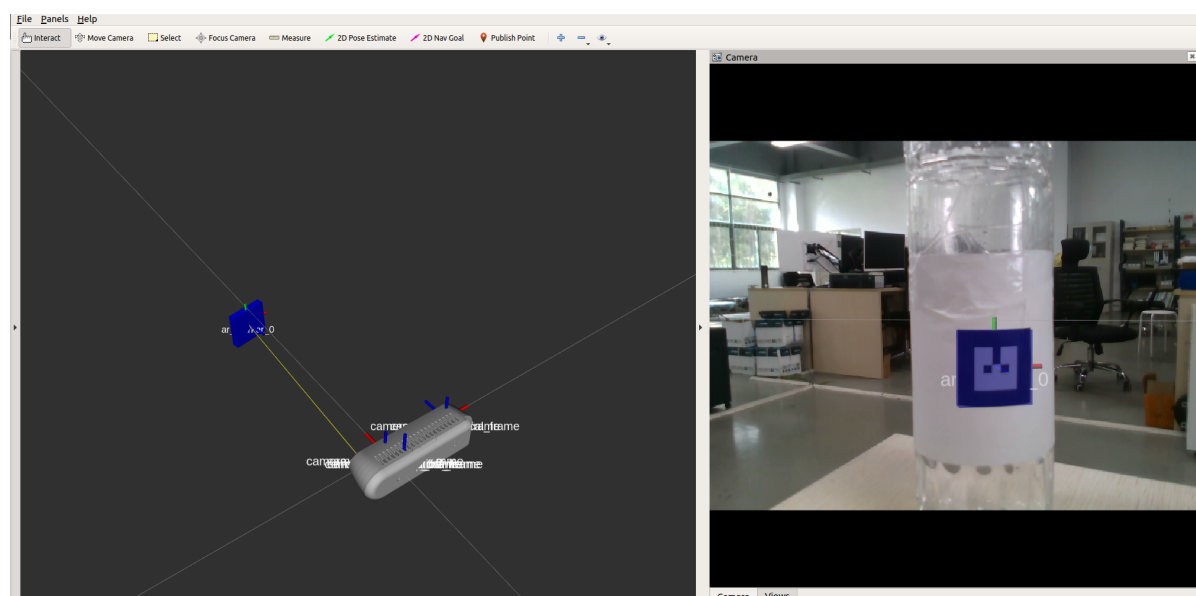
  <group if="$(eval camera_model == 'realsense_d435')">
    <include file="$(find realsense2_camera)/launch/rs_camera.launch">
      <arg name="camera"           value="$(arg camera_namespace)"/>
      <arg name="enable_pointcloud" value="false" />
      <arg name="serial_no"       value="$(arg serial_no)"/>
    </include>

    <node unless="$(arg use_quaternion)" pkg="tf" type="static_transform_publisher" name="$(arg camera_namespace)_to_realsense_frame"
      args="$(arg x) $(arg y) $(arg z) $(arg yaw) $(arg pitch) $(arg roll) $(arg parent_link) $(arg camera_namespace)_link 10" />

    <node if="$(arg use_quaternion)" pkg="tf" type="static_transform_publisher" name="$(arg camera_namespace)_to_realsense_frame"
      args="$(arg x) $(arg y) $(arg z) $(arg r_x) $(arg r_y) $(arg r_z) $(arg r_w) $(arg parent_link) $(arg camera_namespace)_link 10" />

    <include file="$(find ar_track_alvar)/launch/pr2_indiv_no_kinect.launch">
      <arg name="marker_size" value="$(arg user_marker_size)" />
      <arg name="max_new_marker_error" value="0.08" />
      <arg name="max_track_error" value="0.2" />
      <arg name="cam_image_topic" value="$(arg camera_namespace)/color/image_raw" />
      <arg name="cam_info_topic" value="$(arg camera_namespace)/color/camera_info" />
      <arg name="output_frame" value="$(arg camera_namespace)$(arg marker_frame_id)" />
      <arg name="node_name" value="$(arg camera_namespace)"/>
    </include>
  </group>
</launch>
```

参数	默认值	功能
x,y,z	0	摄像头固定在机械臂中的静态位置
roll,pitch,yaw	0	摄像头固定在机械臂中的静态姿态欧拉角表示形式
r_w,r_x,r_y,r_z	0	摄像头固定在机械臂中的静态姿态四元数表示形式
parent_link	wrist3_Link	摄像头发布tf的相对坐标系
serial_no	空	启动摄像头的序列号
user_marker_size	3	标签的尺寸大小（cm）
use_quaternion	false	是否用四元数表示姿态
camera_model	realsense_d435	摄像头模型
camera_namespace	camera	摄像头命名空间



3.4 使用Gmapping构建地图

3.4.1 功能简介

gmapping功能包订阅机器人的深度信息、IMU信息和里程计信息，同时完成一些必要参数的配置，即可创建并输出基于概率的二维栅格地图。gmapping功能包基于openslam社区的开源SLAM算法。

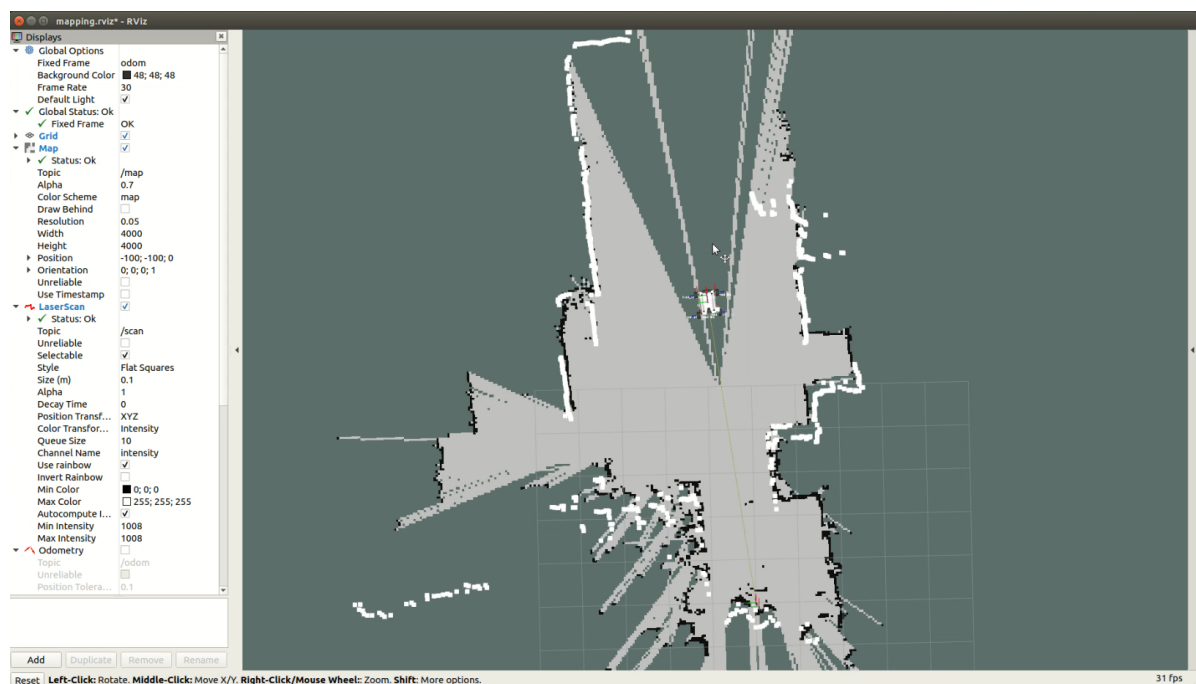
3.4.2 功能运行

(1) 启动雷达

```
roslaunch agilex_ranger open_lidar.launch
```

(2) 启动建图

```
roslaunch agilex_ranger gmapping.launch
```



构建好地图之后，把地图保存在~/catkin_workspace/src/agilex_ranger/maps目录下

(1) 进入~/catkin_workspace/src/agilex_ranger/maps目录

```
roscd agilex_ranger/maps
```

(2) 运行保存地图的命令

```
roslaunch map_server map_saver -f map
```

最后面的map是生成地图的名字，在导航的时候默认是加载以map命名地图文件，如果在命名的时候取了别名字需要修改加载地图的名字。把~/catkin_workspace/src/agilex_ranger/launch/navigation_4wd.launch文件中下面地图名字改为你命名的名字。

```
<node name="map_server" pkg="map_server" type="map_server" args="$(find agilex_ranger)/maps/map.yaml" output="screen">
```

3.5 使用rtabmap算法构建地图

3.5.1 功能简介

rtabmap算法提供一个与时间和尺度无关的基于外观的定位与构图解决方案。针对解决大型环境中的在线闭环检测问题。方案的思想在于为了满足实时性的一些限制，闭环检测是仅仅利用有限数量的一些定位点，同时在需要的时候又能够访问到整个地图的定位点。

3.5.2 功能运行

```
$ roslaunch agilexpro open_lidar.launch
$ roslaunch agilexpro open_camera2.launch
$ roslaunch agilexpro rtabmapping.launch
```

运行rviz，将数据可视化

```
$ rviz
```

3.6 使用move_base导航

3.6.1 功能简介

move_base 包提供了一个动作的实现（参见 actionlib 包），给定世界上的目标，它将尝试使用移动基地来实现它。move_base 节点将全局和本地规划器链接在一起以完成其全局导航任务。move_base 节点还维护两个成本地图，一个用于全局规划器，另一个用于本地规划器（参见 costmap_2d 包），用于完成导航任务。

3.6.2 功能运行

（1）启动激光雷达

```
roslaunch agilex_ranger open_lidar.launch
```

（2）启动move_base导航

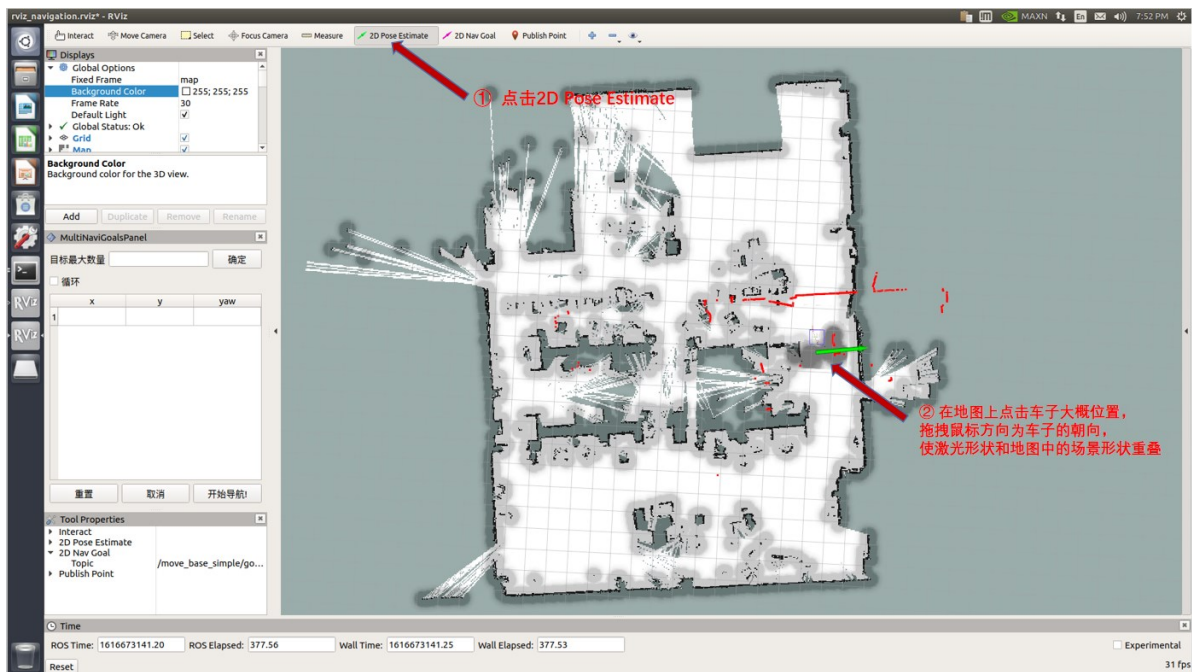
启动导航之前，需要使能can模块

```
roslaunch ranger_bringup setup_can2usb.bash
```

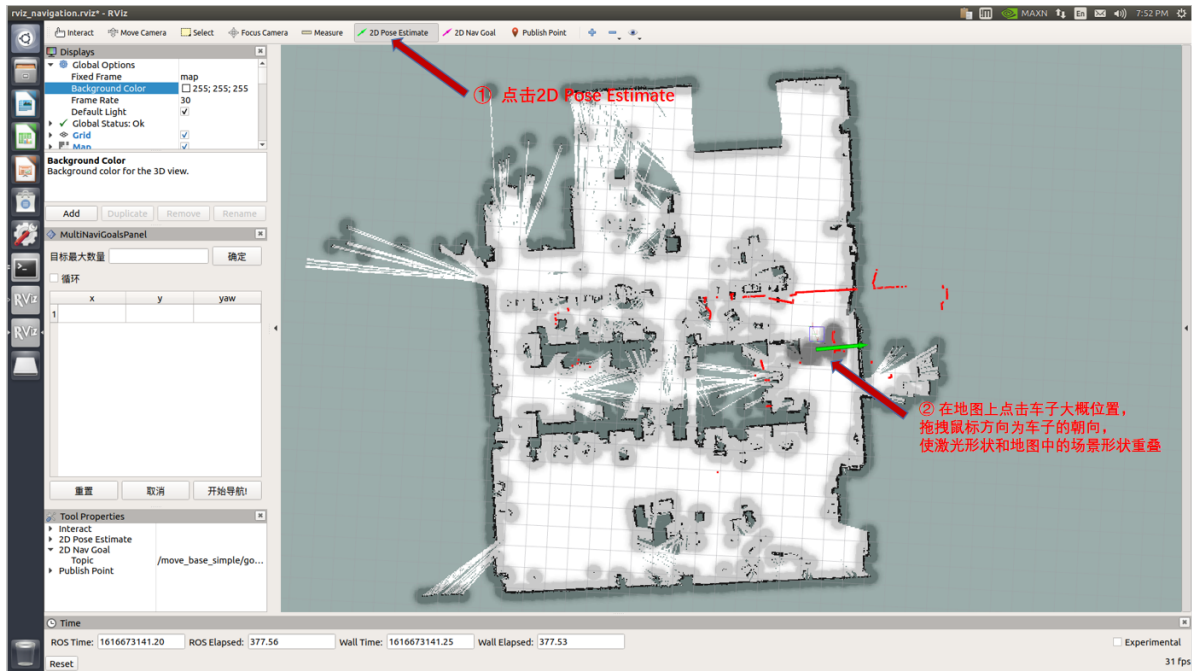
（3）启动move_base

```
roslaunch agilex_ranger navigation.launch
```

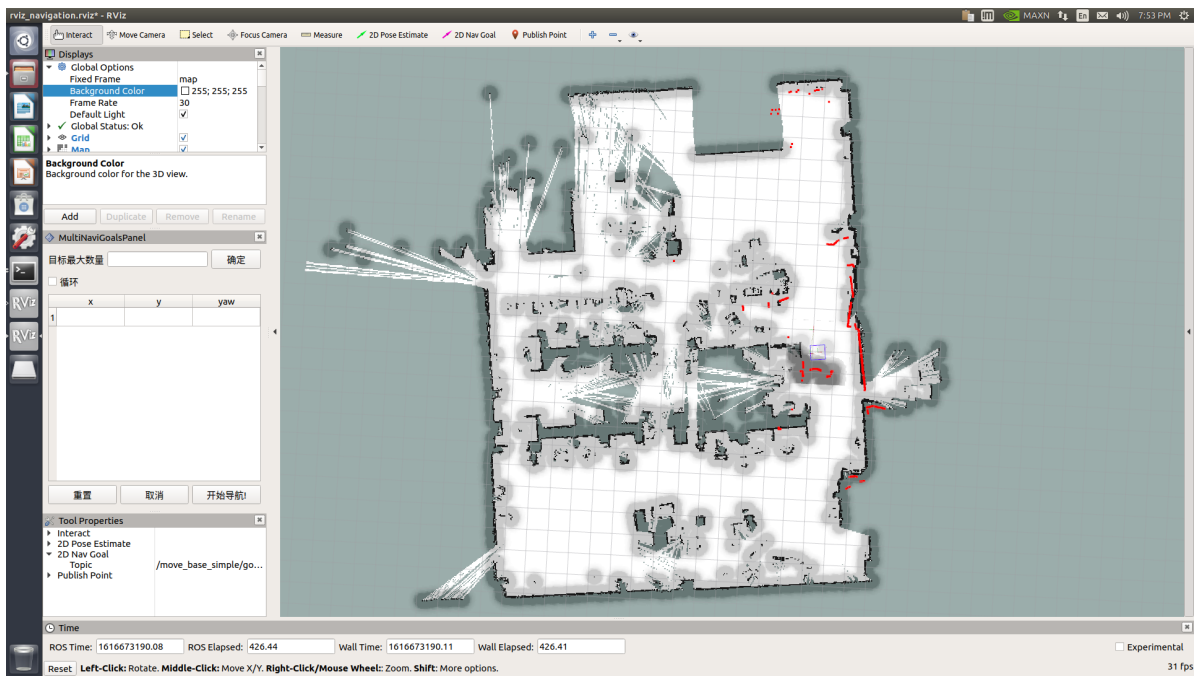
注：如需自定义打开的地图，请打开 navigation_4wd.launch 文件修改参数，如下图所示，请在标记横线处修改为需要打开的地图名称。



(3) 在rviz中显示的地图上矫正底盘在场景中实际的位置，通过发布一个大概的位置加上手柄是底盘旋转校正。当激光形状和地图中的场景形状重叠的时候，校正完成



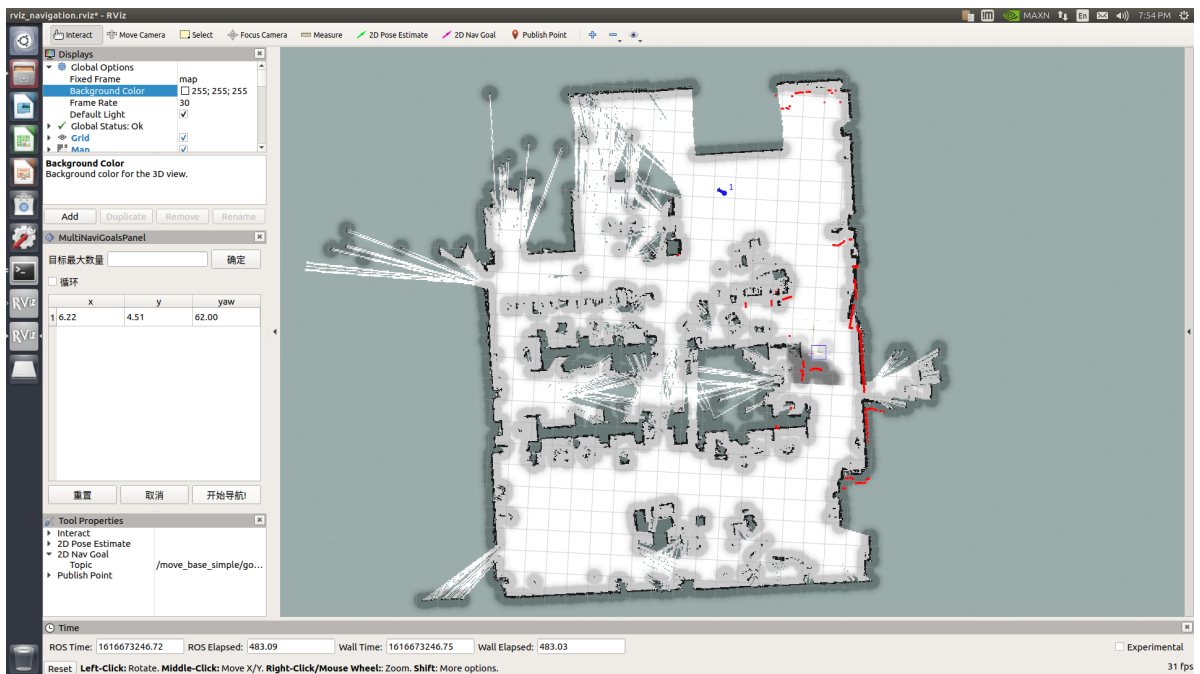
设置定位后，激光形状和地图中的场景形状已基本重合，校正完成。如下图所示



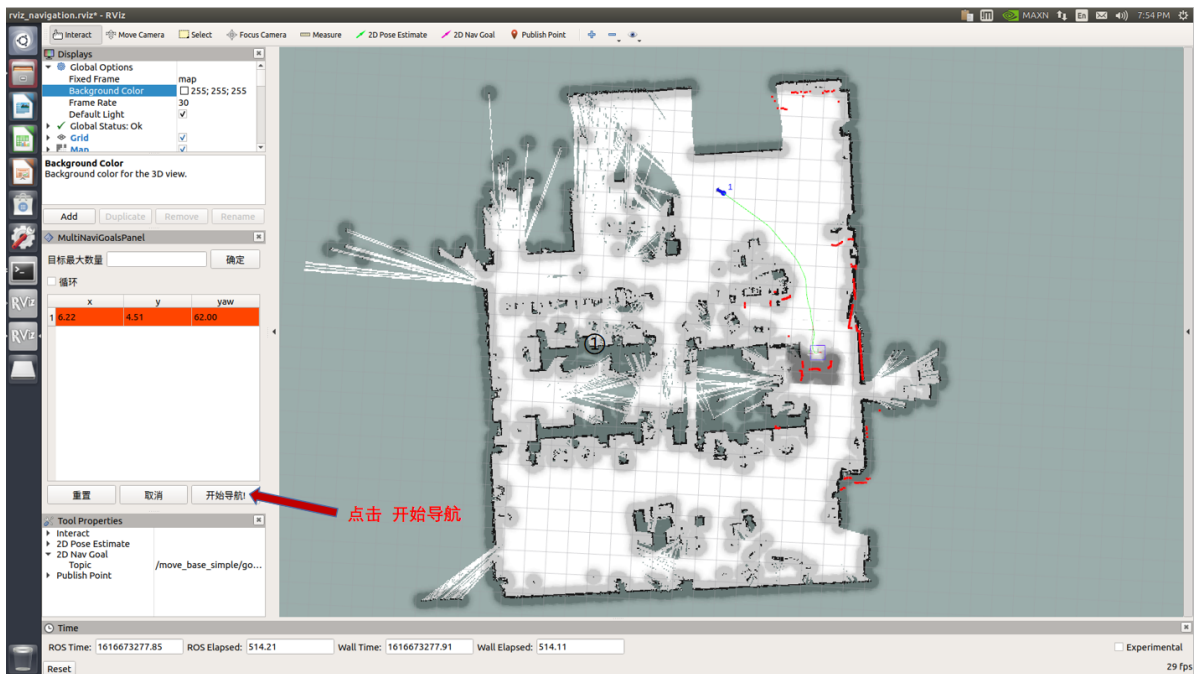
(4) 在左侧栏多点导航的插件上设置目标点，点击开始导航，切换手柄为指令控制模式。



已生成目标点



点击开始导航，地图已生成路径（绿色），将自动导航到目标点



启动雷达、相机和rtabmap算法导航

```
$ roslaunch agilexpro open_lidar.launch
$ roslaunch agilexpro open_camera2.launch
$ roslaunch agilexpro rtabmapping.launch localization:=true
$ roslaunch agilexpro rtabmap_navigation.launch
```

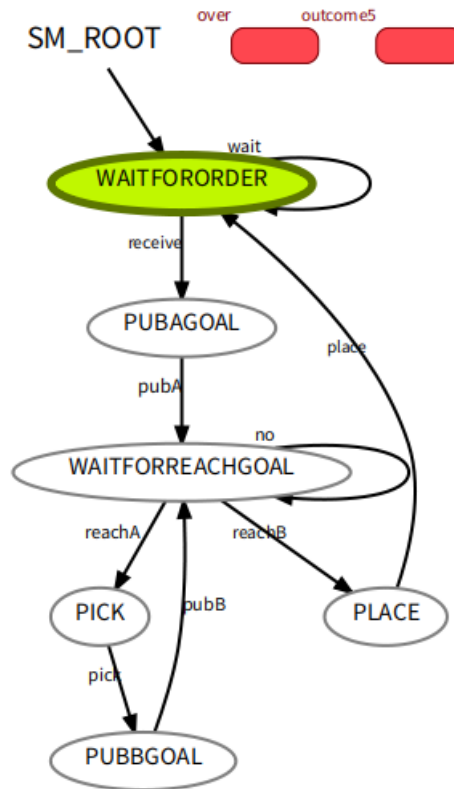
3.7 使用smach库实现任务状态切换

3.7.1功能简介

SMACH中明确描述所有可能的状态和状态转换，在机器人执行一些复杂的计划时，很有用。这基本上消除了将不同模块组合在一起的冲突，使移动机器人操控等系统做更多有趣的事情

3.7.2功能运行

```
$ rosrun agx_cr5_smach smach_demo.py
$ rosrun smach_viewer smach_viewer.py
```



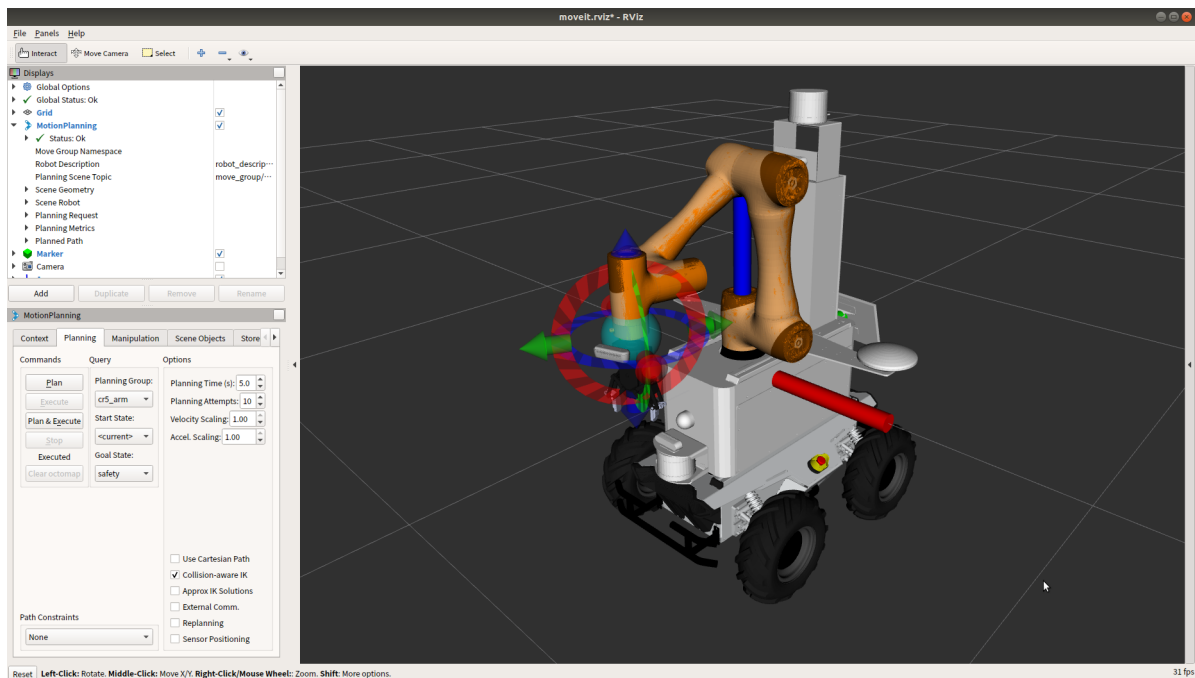
3.8 使用MoveIt! 控制移动+臂仿真模型

3.8.1 功能简介

利用MoveIt!成熟的运动规划器对机械臂的运动进行路径规划、碰撞检测和路径执行。从而在给定目标点的时候可以计算出一条最优路径出来。

3.8.2 功能运行

```
$ roslaunch scout_cr5_moveit_config demo.launch
```

这是完整的底盘与机械臂的组合模型，我们可以拖动机械臂末端执行路径规划，此时的机械臂不会与底盘发生碰撞。

4 功能使用

4.1 启动机械臂

启动机械臂之前需要先把机械臂遥控出打包姿态，比如遥控到零姿态

```
$ roslaunch agilex_cr5 bringup_arm.launch
```

参考3.2连接控制机械臂运动使机械臂正常上电使能，该launch文件会同时启动机械臂控制驱动节点和Moveit！根据移动底盘和机械臂的符合URDF模型控制真实机械臂运动。

在这个launch file里面可支持修改一个参数，robot_ip。这里传入的ip为机械臂控制柜默认ip：192.168.8.188。

```
<?xml version='1'?>
<launch>
  <arg name="robot_ip" default="192.168.8.188" />
  <include file="$(find dobot_bringup)/launch/bringup.launch">
    <arg name="robot_ip" value="$(arg robot_ip)" />
  </include>

  <include file="$(find scout_cr5_moveit_config)/launch/cr5_moveit.launch" />
</launch>
```

4.2 启动摄像头

```
$ roslaunch agilex_cr5 open_camera.launch
```

注意启动之前需要用USB3.0的数据线连接摄像头与工控机。

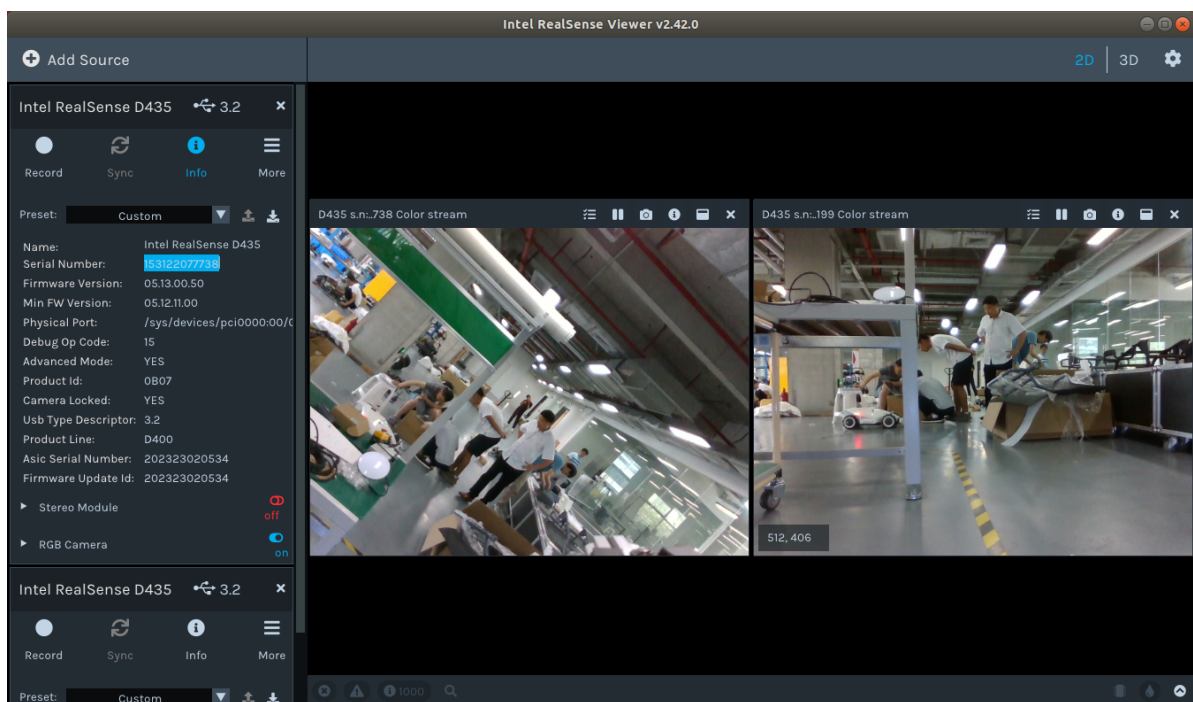
该launch file支持修改的参数有下面几个：

```

<arg name="camera_namespace" value="cam1"/>
<arg name="serial_no" value="153122077738"/> <!--153122077738 135622071199 相机
序列号可通过realsense-viewer查看-->
<arg name="roll" value="0"/>
<arg name="pitch" value="-1.5708"/>
<arg name="yaw" value="0.7854"/>
<arg name="x" value="-0.072"/>
<arg name="y" value="-0.045"/> <!---0.02-->
<arg name="z" value="0.01"/>
<arg name="use_quaternion" value="false"/>
<arg name="parent_link" value="Link6"/>

```

- camera_namespace: 摄像头节点的命名空间, 当启动多个摄像头时可以需要修改每个摄像头的命名空间。
- serial_no: 摄像头的序列号S/N码, 当需要启动多个摄像头的时候可以传入摄像头的序列号指定打开某个摄像头。



realsense-viewer查看相机的Serial Number。

- use_quaternion: true表示使用四元数, false表示使用欧拉角。
- x,y,z: 摄像头距离机械臂的位置。
- roll,pitch,yaw: 摄像头姿态欧拉角。
- r_x,r_y,r_z,r_w: 摄像头姿态四元数。

在启动这个launch file的时候也启动了图像识别节点, 会实时更新物体和物体摆放平台的空间坐标信息, 可根据以下接口获取这两点信息。服务名字为 /pick_point。服务类型为 agx_pick_msg/AgxPickSrv, 详细的消息格式如下:

```

int32 item           # 0--bottle 1--pick 2--place
int32 handpose       # 夹爪夹取物体的方向 0--front 1--upside
int32 Prepose        # 预备夹取点.0--物体真实位置;
                     # 1--根据handpose往前或者往上偏移一个夹爪的位置
---
#返回值

```

```

geometry_msgs/Point position
  float64 x
  float64 y
  float64 z
geometry_msgs/Quaternion orientation
  float64 x
  float64 y
  float64 z
  float64 w

```

其中q为0时，返回物体的坐标信息；q为2时，返回物体摆放位置信息。

4.3 启动机械爪

```
$ roslaunch agilex_cr5 open_gripper.launch
```

控制机械爪的topic接口为/gripper/ctrl,消息的类型为dh_gripper_msgs/GripperCtrl，消息格式如下：

```

bool initialize          #是否初始化
float32 position         #两个夹片的间距
float32 force            #夹取物体的力度
float32 speed            #二指夹爪这个参数无效，夹取速度和force正相关

```

在agx_cr5_pick功能包里面有个demo文件可以输入指尖位置来控制机械爪。使用指令如下：

```
$ rosrun agx_cr5_pick control_gripper #q退出
```

```

zsq@zsq-ThinkPad-E14:~/agile_ws$ rosrun agx_aubo_pick control_gripper
1 open ; 0 close

```

4.4 启动agx_cr5_pick节点

```
$ roslaunch agilex_cr5 agx_cr5_pick.launch
```

该节点需要获取状态机传入的taskid来分别控制机器人执行不同的任务。任务的服务接口为/send_task，服务类型为agx_pick_msg/TaskCmd，消息格式如下：

```

int32 taskID
geometry_msgs/PoseStamped A_goal
  std_msgs/Header header
    uint32 seq
    time stamp
    string frame_id
  geometry_msgs/Pose pose
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
  geometry_msgs/PoseStamped B_goal

```

```

std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
geometry_msgs/Pose pose
geometry_msgs/Point position
  float64 x
  float64 y
  float64 z
geometry_msgs/Quaternion orientation
  float64 x
  float64 y
  float64 z
  float64 w
---
bool result

```

4.5 启动smach状态机节点

```
$ rosrun agx_cr5_smach smach_demo.py
```

定义了以下几种状态的转换：

```

smach.StateMachine.add('WAITFORORDER', WaitFororder(),
    transitions={'receive':'PUBAGOAL',
    'wait':'WAITFORORDER'})

smach.StateMachine.add('PUBAGOAL', PubAGoal(),
    transitions={'pubA':'WAITFORREACHGOAL'})

smach.StateMachine.add('WAITFORREACHGOAL', WaitForReachGoal(),
    transitions={'no':'WAITFORREACHGOAL',
    'reachA':'PICK',
    'reachB':'PLACE'})

smach.StateMachine.add('PICK', Pick(),
    transitions={'pick':'PUBBGOAL'})

smach.StateMachine.add('PUBBGOAL', PubBGoal(),
    transitions={'pubB':'WAITFORREACHGOAL'})

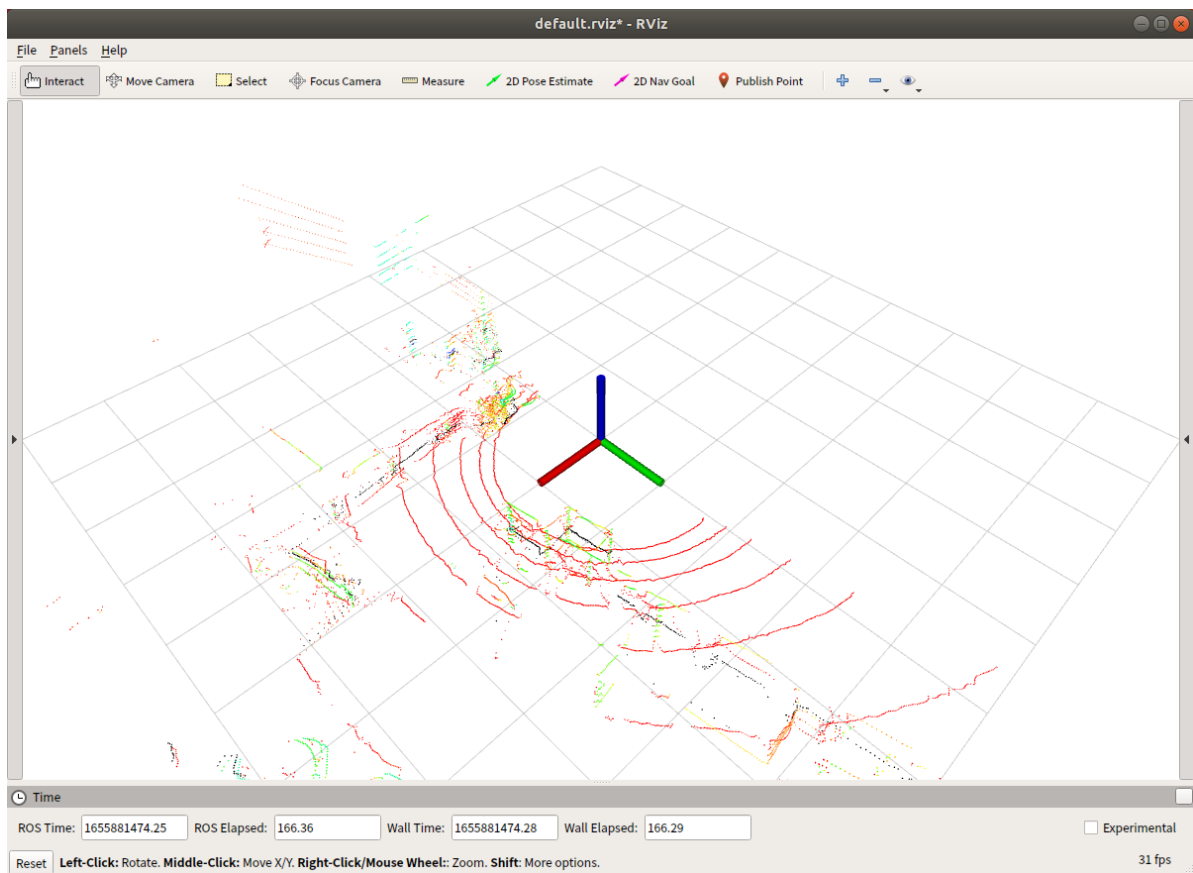
smach.StateMachine.add('PLACE', Place(),
    transitions={'place':'WAITFORORDER'})

```

以上是整个系统的状态转换逻辑，可以根据需要添加相应的状态。以第一个状态为例，在这里定义了一个“WAITFORORDER”的状态，这个状态的类为WaitFororder，transitions代表状态的跳转。如果WaitFororder输出的结果为receive则跳转到“PUBAGOAL”，如果状态输出结果为wait，则跳转到“WAITFORORDER”状态。

4.6 启动雷达

```
$ roslaunch agilex_ranger open_lidar.launch
```



4.7 启动建图

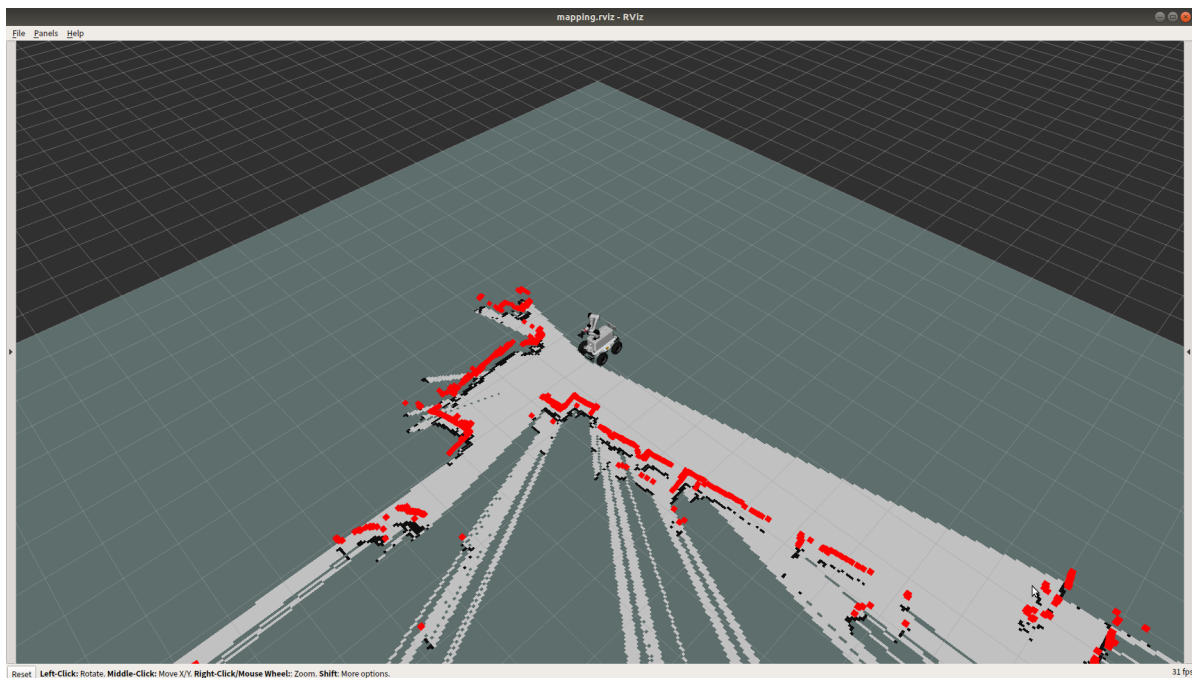
gmapping算法建图

(1) 启动雷达

```
roslaunch agilex_ranger open_lidar.launch
```

(2) 启动gmapping建图

```
roslaunch agilex_ranger gmapping.launch
```



用遥控器遥控机器人在场景中建图，执行以下指令保存地图

(1) 进入~/catkin_workspace/src/agilex_ranger/maps目录

```
roscd agilex_ranger/maps
```

(2) 运行保存地图的命令

```
roslaunch map_server map_saver -f map
```

最后面的map是生成地图的名字，在导航的时候默认是加载以map命名地图文件，如果在命名的时候取了别名字需要修改加载地图的名字。把

~/catkin_workspace/src/agilex_ranger/launch/navigation_4wd.launch文件中下面地图名字改为你命名的名字。

```
<node name="map_server" pkg="map_server" type="map_server" args="$(find
agilex_ranger)/maps/map.yaml" output="screen">
```

使用rtabmap算法构建地图

功能运行

```
$ roslaunch agilexpro open_lidar.launch
$ roslaunch agilexpro open_camera2.launch
$ roslaunch agilexpro rtabmapping.launch
```

运行rviz，将数据可视化

```
$ rviz
```

4.8 启动导航

激光导航

(1) 启动激光雷达

```
roslaunch agilex_ranger open_lidar.launch
```

(2) 启动move_base导航

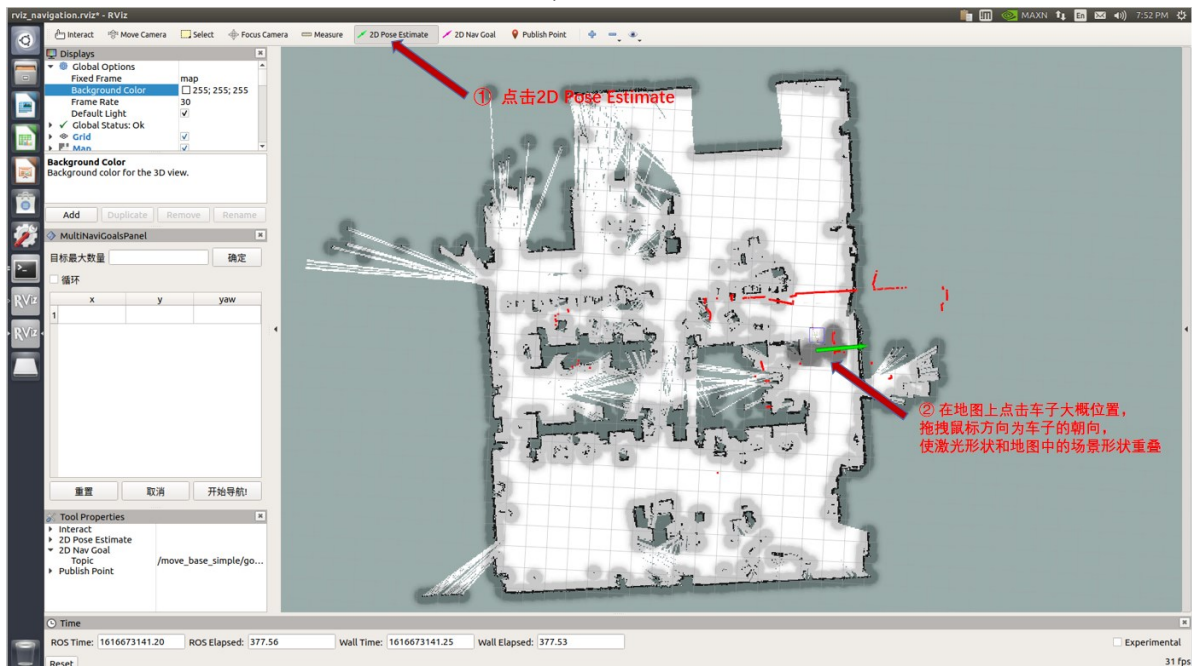
启动导航之前，需要使能can模块

```
roslaunch scout_bringup setup_can2usb.bash
```

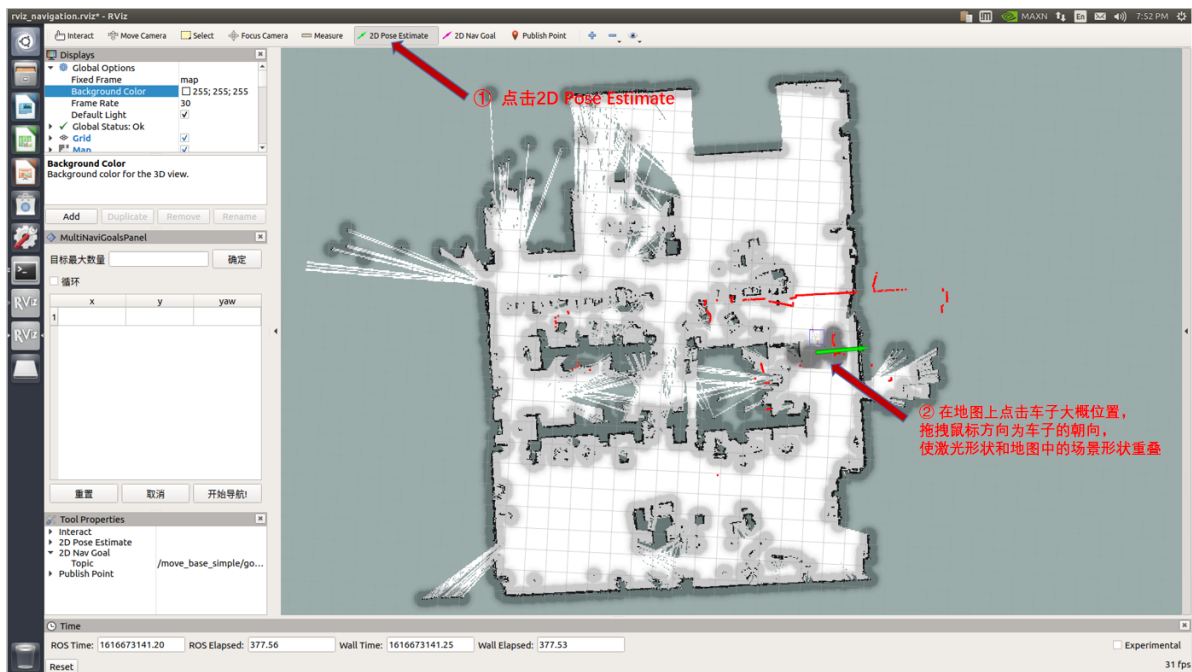
(3) 启动move_base

```
roslaunch agilex_ranger navigation.launch
```

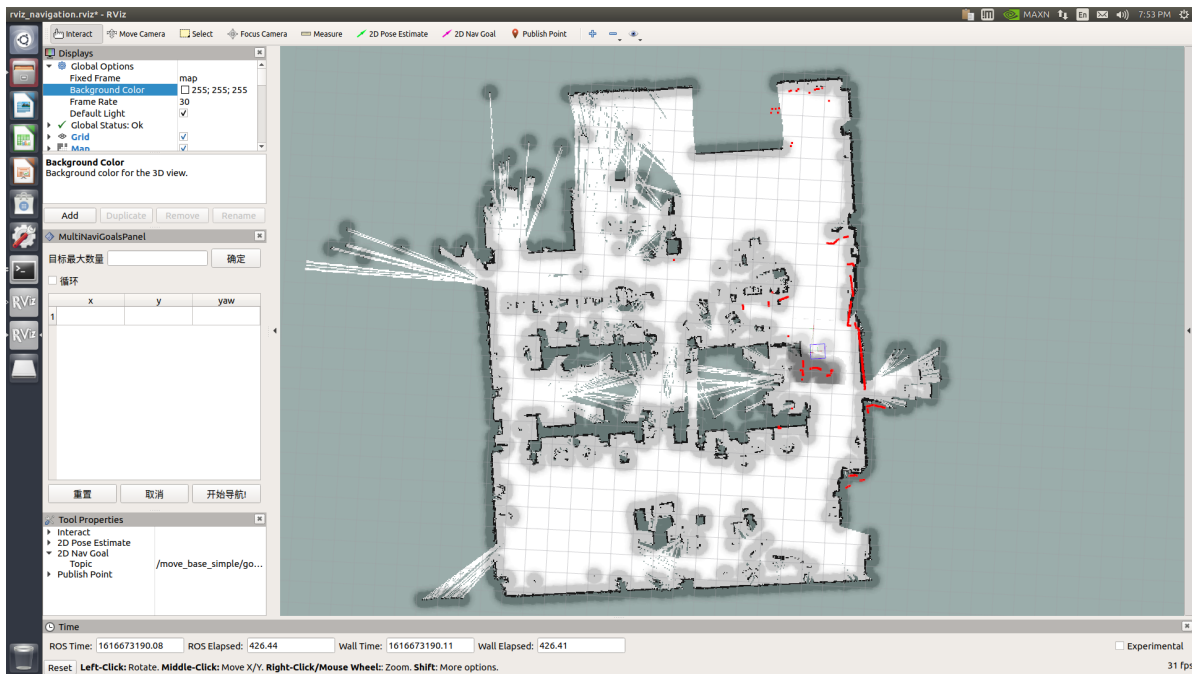
刚刚启动的时，程序默认车子在启动建图的位置，需要通过发布一个大概的位置加上手柄旋转底盘校正。当激光形状和地图中的场景形状重叠的时候,校正完成。



(3) 在rviz中显示的地图上校正底盘在场景中实际的位置，通过发布一个大概的位置加上手柄是底盘旋转校正。当激光形状和地图中的场景形状重叠的时候，校正完成



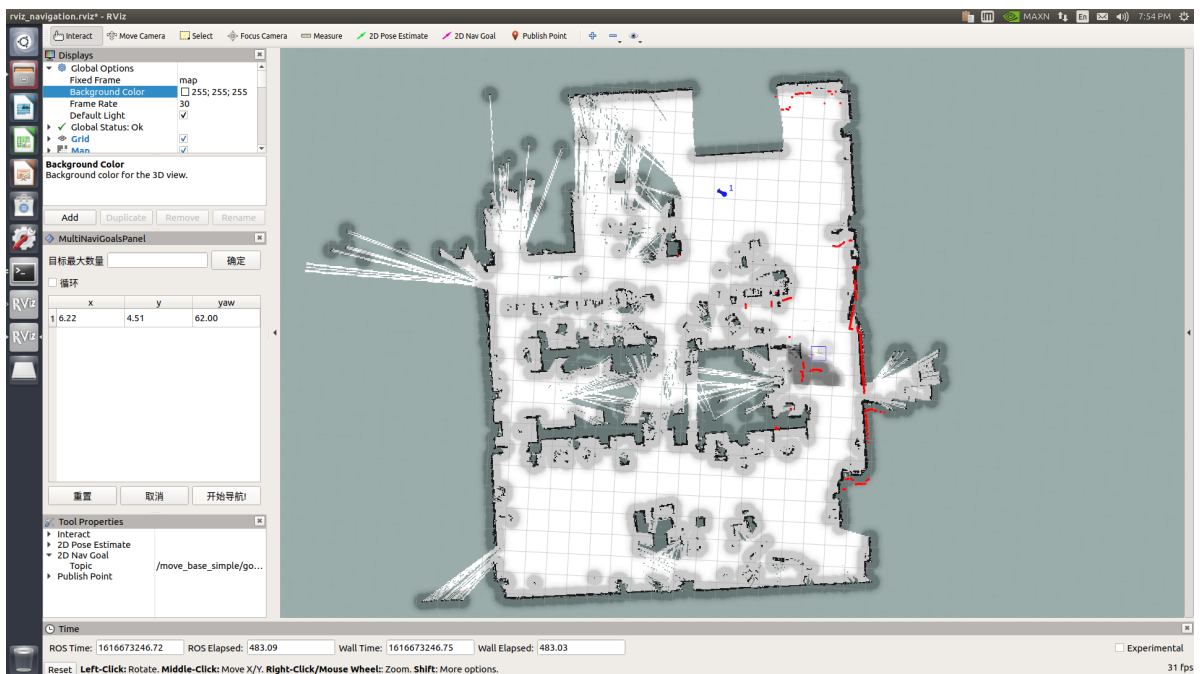
设置定位后，激光形状和地图中的场景形状已基本重合，校正完成。如下图所示



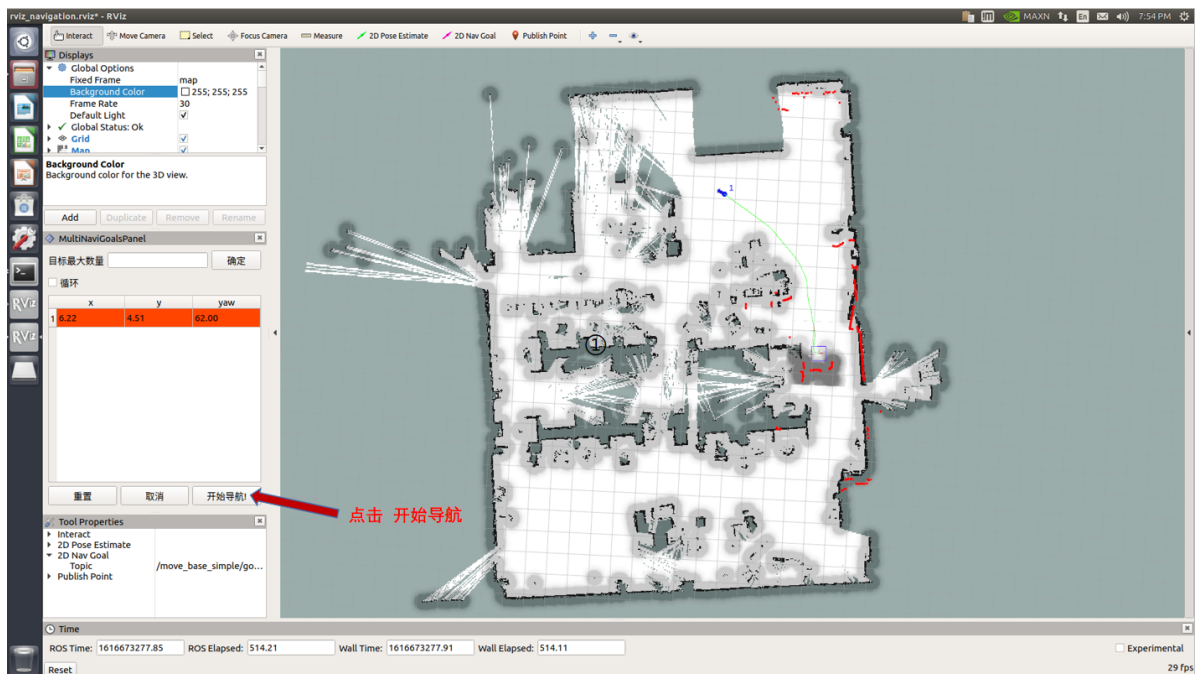
(4) 在左侧栏多点导航的插件上设置目标点，点击开始导航，切换手柄为指令控制模式。



已生成目标点



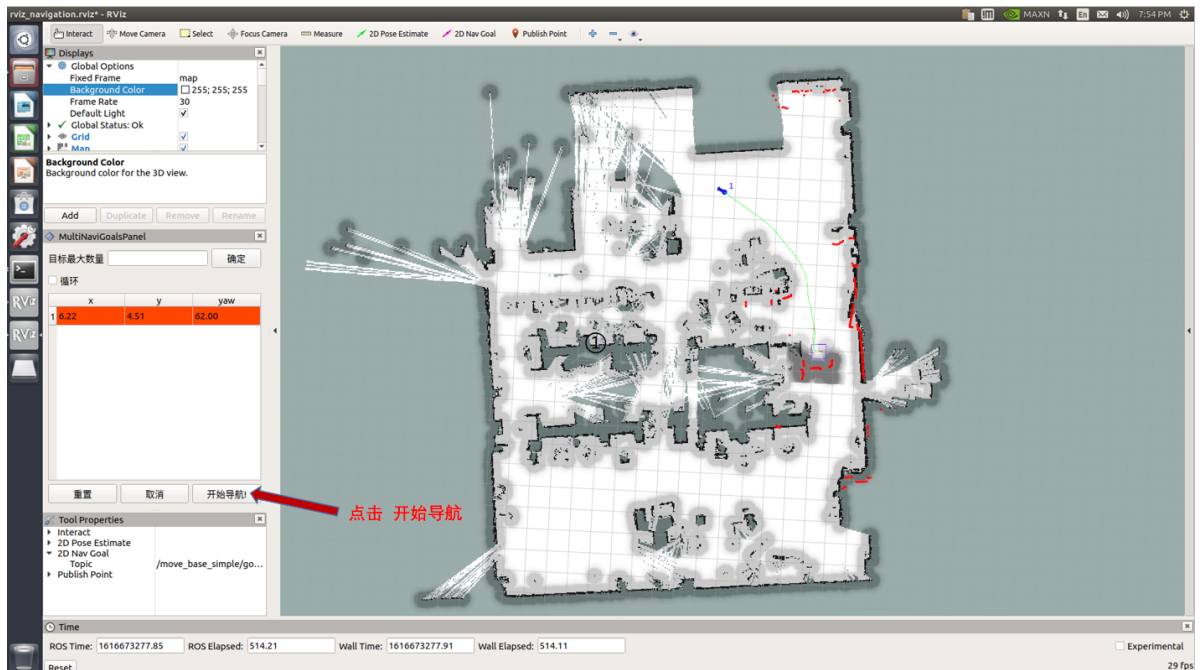
点击开始导航，地图已生成路径（绿色），将自动导航到目标点



启动雷达、相机和rtabmap算法导航

```
$ roslaunch agilexpro open_lidar.launch
$ roslaunch agilexpro open_camera2.launch
$ roslaunch agilexpro rtabmapping.launch localization:=true
$ roslaunch agilexpro rtabmap_navigation.launch
```

刚刚启动的时，程序默认车子在启动建图的位置，手柄旋转底盘校正。当激光形状和地图中的场景形状重叠的时候,校正完成。



导航的某点话题接口为/move_base_simple/goal，消息类型为geometry_msgs/PoseStamped，消息格式如下：

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
geometry_msgs/Pose pose
```

```
geometry_msgs/Point position
  float64 x
  float64 y
  float64 z
geometry_msgs/Quaternion orientation
  float64 x
  float64 y
  float64 z
  float64 w
```

到达某个点之后的反馈信息可以从/move_base/result话题获得，消息类型为base_msgs/MoveBaseActionResult，消息格式如下：

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
actionlib_msgs/GoalStatus status
  uint8 PENDING=0
  uint8 ACTIVE=1
  uint8 PREEMPTED=2
  uint8 SUCCEEDED=3
  uint8 ABORTED=4
  uint8 REJECTED=5
  uint8 PREEMPTING=6
  uint8 RECALLING=7
  uint8 RECALLED=8
  uint8 LOST=9
actionlib_msgs/GoalID goal_id
  time stamp
  string id
  uint8 status
  string text
move_base_msgs/MoveBaseResult result
```

当机器人到达目标点后可以查看status为3。

4.9 录制两导航点的坐标位置

```
$ roscd agx_cr5_pick/config/
$ rosrn agx_cr5_pick record
```

(1) 进入保存坐标信息的目录

```
$ roscd agx_cr5_pick/config/
```

(2) 启动雷达和导航

```
$ roslaunch agilex_ranger open_lidar.launch
$ roslaunch agilexpro open_camera2.launch
$ roslaunch agilexpro rtabmapping.launch localization:=true
```

(3) 把小车移动到指定的位置，比如工件面前，大约1m的位置

(4) 启动录制节点，按下回车记录两个点的位置。

```
roslaunch agx_cr5_pick record
```

4.10 移动抓取演示

在场景中设置好两个地点作为一个夹取点和摆放点，通过步骤9录制好两个地点的坐标位置之后，开启以下指令。

```
$ roslaunch agilex_cr5 bringup_arm.launch           #启动机械臂
$ roslaunch agilex_cr5 open_camera.launch           #启动相机
$ roslaunch agilex_cr5 open_gripper.launch          #启动夹爪
$ roslaunch agilex_cr5 agx_cr5_pick.launch          #启动移动夹取节点
$ roslaunch agilex_cr5 smach smach_demo.py          #启动状态机
$ roslaunch agilex_ranger open_lidar.launch         #启动雷达
$ roslaunch agilex_ranger navigation_4wd.launch     #启动导航
$ roslaunch agx_cr5_pick test.launch               #开始执行任务
```

切换遥控器为指令控制模式，则复合移动机器人开始导航到第一个地点抓取东西。

4.11 原地抓取和放置演示

把抓取物体贴上标签，摆放到机器人一个合适的位置。执行以下指令就可以抓取和摆放的演示

```
$ roslaunch agilex_cr5 bringup_arm.launch
$ roslaunch agilex_cr5 open_camera.launch
$ roslaunch agilex_cr5 open_gripper.launch
$ roslaunch agx_cr5_pick demo_pick                 #抓取物体
$ roslaunch agx_cr5_pick demo_place                 #放置物体
```

5 常见问题的处理与解答

6 其他说明

1. [Dobot CR5 硬件使用手册V2.3](#)
2. https://github.com/Dobot-Arm/CR_ROS
3. [机械臂客户端下载](#)
4. [SCOUT用户手册](#)
5. [AgileX GitHub](#)
6. [RS-LiDAR-16用户手册](#)
7. [大寰手爪操作手册](#)

